

The Name of the Title Is Hope

Anonymous Author(s)

Abstract

Automated unit test generation is essential for improving software reliability, yet existing techniques struggle to handle complex program semantics, especially when path reachability depends on implicit object states and inter-procedural interactions. Recent large language model (LLM)-based approaches can directly generate executable tests, but often fail to satisfy hidden preconditions that are not locally visible, resulting in low coverage. In this paper, we propose COGPATH, an LLM-driven framework that improves test generation by making implicit constraints explicit and reducing irrelevant context. Specifically, we introduce *Constraints-Hints* to expose path conditions as structured input requirements, and a coverage-driven backward slicing technique to retain only statements relevant to target branches. These components are integrated into an iterative pipeline that combines generation, execution, and repair. Experimental results on real-world Java projects show that COGPATH consistently achieves higher branch coverage, particularly for paths requiring precise object state construction.

Keywords

Automated Unit Test Generation, Large Language Models, Software Testing, Program Analysis, Branch Coverage, Constraint-Guided Testing, Program Slicing

ACM Reference Format:

Anonymous Author(s). 2018. The Name of the Title Is Hope. *J. ACM* 37, 4, Article 111 (August 2018), 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Automated unit test generation plays a critical role in improving software reliability and reducing developer effort. Traditional techniques, such as symbolic execution and search-based software testing (SBST), have made substantial progress in systematically exploring input spaces and maximizing structural coverage [2, 5, 11]. However, these approaches often face fundamental limitations in scalability and in handling complex program semantics. In particular, they struggle to capture implicit preconditions that arise from intricate object states and inter-procedural interactions, where the conditions required to reach a target path are established outside the local method scope through sequences of prior method calls [1].

Recent advances in large language models (LLMs) have introduced a new paradigm for unit test generation. By leveraging their strong code understanding and reasoning capabilities, LLM-based approaches can directly synthesize executable test cases from source code, significantly lowering the barrier to automated testing [3, 7,

9, 14, 18]. Moreover, iterative workflows that incorporate compilation and execution feedback have further improved the validity and coverage of generated tests [13]. [xtz:lack detailed description of several representative related work.]

Despite these promising results, existing LLM-based methods still exhibit a fundamental limitation: they lack precise reasoning about *implicit program constraints*, particularly those that span across multiple methods. Most approaches rely on prompting the LLM with the entire method body, optionally augmented with control-flow or type information. However, in real-world object-oriented programs, the conditions required to execute a target branch are often not locally visible within the method under test. Instead, they depend on object states established through prior method calls or initialization sequences, which are external to the focal method. As a result, LLMs frequently generate tests that are syntactically correct but fail to satisfy the hidden preconditions necessary for reaching uncovered branches [14]. [xtz: Despite these promising results, a substantial coverage gap still remains, especially on complex branches whose reachability depends on non-local object states and long dependency chains. This suggests that the remaining challenge is not simply a lack of feedback or prompting sophistication, but a deeper mismatch between the information exposed to the LLM and the information actually required to cover a target branch. ① One important dimension of this mismatch is *what to satisfy*. In many cases, branch reachability depends on implicit preconditions scattered across long dependency chains, requiring the model to infer hidden constraints through multiple variable assignments and state transitions before it can synthesize a valid test. Existing approaches typically provide only the target branch structure and compress constraint inference and test synthesis into a single generation step, without any explicit intermediate output or independent verification of the inferred constraints. As a result, the LLM can easily skip or compress critical reasoning steps, producing tests that appear plausible but fail to satisfy the actual conditions required to reach the target branch. This problem becomes particularly severe when the underlying dependency chain is long or semantically complex. ② The second dimension concerns *what to attend to*. Existing methods usually expose the entire method body to the LLM, even though only a small subset of statements has real data or control dependence on the target branch. The resulting context is therefore structurally misaligned with the branch-coverage objective: coverage-critical dependencies are diluted by irrelevant code, attention is spread across non-essential statements, and the model must reason under a representation that over-emphasizes what is visible rather than what is truly necessary. This not only amplifies cognitive burden, but also makes precise state construction substantially less likely. Even without explicit distractors, LLM performance can already degrade as context length grows [4, 17]; in branch-oriented test generation, this effect is further magnified because the relevant context is often deeply buried inside much longer method bodies. Taken together,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM 1557-735X/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

these two issues suggest a systematic form of *cognitive misalignment* in LLM-based unit test generation: the information shown to the model is often misaligned with the information actually needed to cover a target branch. This misalignment has two complementary dimensions, namely *what to satisfy* and *what to attend to*.]

Furthermore, providing the full method body as context introduces substantial noise. Irrelevant statements consume the limited context window and obscure the critical dependencies that govern branch reachability. This not only increases the cognitive burden on the LLM but also reduces its ability to infer correct execution conditions. Consequently, even when guided by path information, existing approaches often fail to cover branches that require precise state construction, as LLM performance has been shown to degrade significantly as input context length increases, even in the absence of distracting information [4, 17].

To address these challenges, we propose COGPATH, a novel LLM-driven framework for automated unit test generation that tightly integrates program analysis with language model reasoning. The key idea is to reduce the difficulty of test generation from two complementary perspectives: *making hidden constraints explicit and eliminating irrelevant context*. [xtz: To address this cognitive misalignment, we propose COGPATH, an LLM-driven framework for automated unit test generation that explicitly tackles both dimensions of the gap, by making critical constraints explicit and concentrating the model's attention on branch-relevant program context. First, *Constraints-Hints* addresses the *what to satisfy* dimension. It extracts path conditions from the control-flow graph and externalizes them as explicit, human-interpretable input requirements prior to test synthesis, turning implicit reachability constraints into a structured intermediate artifact that directly guides code generation. This decoupling ensures that preconditions buried in long dependency chains are surfaced and reasoned about explicitly, rather than silently skipped in a single end-to-end generation step. Second, *coverage-driven backward slicing* addresses the *what to attend to* dimension. Rather than supplying the full method body, the slicer isolates the minimal set of statements that genuinely influence an uncovered branch, including cross-method dependencies such as field initializations and setter methods. This restructures the prompt context around branch-relevant dependencies, preventing the model's attention from being diffused across irrelevant code. The slicing is triggered selectively when coverage plateaus, targeting branches that resist standard generation. Both components are integrated into an iterative loop of generation, execution, and repair. Coverage feedback drives target selection each round, progressively exposing branches where the current context remains misaligned with their reachability requirements, until coverage converges or the budget is exhausted.]

First, we introduce *Constraints-Hints*, a mechanism that extracts path conditions from the control-flow graph and translates them into explicit, human-interpretable input requirements. Instead of requiring the LLM to infer complex branch predicates from raw source code, our approach directly exposes the necessary conditions for covering a target path, enabling more accurate and guided test generation.

Second, we propose a *coverage-driven backward slicing* technique that identifies the minimal subset of program statements relevant

to an uncovered branch. By focusing only on statements that influence the target condition—including cross-method dependencies such as field initialization and setter methods—the resulting slice provides a concise and semantically meaningful context for the LLM.

These two components are integrated into an iterative framework that combines test generation, execution, and automated repair. Starting from an initial test suite, the framework continuously identifies uncovered paths, derives constraint hints, and generates candidate tests. When coverage improvement stagnates, backward slicing is triggered to provide more precise guidance. This iterative feedback loop enables sustained coverage growth and effective exploration of hard-to-reach program paths.

We evaluate COGPATH on a set of real-world Java projects and compare it against state-of-the-art LLM-based and search-based baselines. The results show that our approach consistently achieves higher branch coverage and is particularly effective in scenarios involving complex object state dependencies. Further analysis demonstrates that both constraint hints and backward slicing are essential for guiding LLMs toward generating semantically correct and coverage-effective tests. [xtz: We evaluate COGPATH on 14 Java projects from Defects4J v2.0.1, covering 130 classes and 2971 methods with cyclomatic complexity greater than 10. Baselines include SymPrompt, HITS, and Panta, representing complementary design philosophies. COGPATH achieves 44.41% average branch coverage, outperforming SymPrompt (21.14%), HITS (34.24%), and Panta (32.32%). Gains are most pronounced on logic-heavy subjects with rich cross-method dependencies: on Gson, COGPATH reaches 41.24% versus SymPrompt's 3.59%; on Jsoup, it improves over HITS by 36.4%. Ablation confirms the complementary roles of both components. Removing *Backward Slicing* drops coverage from 44.41% to 38.47%, with the largest degradation on context-heavy subjects (e.g., Csv: 52.29%→24.80%). Removing *Constraint-Hints* drops coverage to 39.79%, most severely on subjects with object-state dependencies (e.g., JacksonDatabind: 17.85%→9.33%). Finally, COGPATH generalizes across backbone LLMs, achieving 44.41% (QC3-30B), 83.47% (QW3.5-397B), and 80.12% (DS-V3.1), confirming that its benefit derives from restructuring branch-relevant information rather than relying on any specific model.] In summary, this paper makes the following contributions:

- We identify a key limitation of existing LLM-based test generation approaches in handling implicit, cross-method constraints.
- We propose *Constraints-Hints*, a mechanism that makes path conditions explicit and guides LLM reasoning.
- We introduce a coverage-driven backward slicing technique to eliminate irrelevant context and expose critical dependencies.
- We design an iterative framework that integrates generation, execution feedback, and repair for sustained coverage improvement.
- We empirically demonstrate the effectiveness of our approach on real-world software systems.

2 MOTIVATION

Recent LLM-based unit test generation approaches, such as SymPrompt [14], HITS [18], Panta [7], and JunitGenie [9], have achieved remarkable progress in automating test generation for complex

methods, significantly reducing the manual effort required from developers. However, when the reachability of a target branch depends on *object states established by prior method calls*, existing approaches still struggle to guide LLMs in constructing the appropriate external context, even when augmented with CFG-based path analysis.

The fundamental limitation is that these approaches typically supply the *entire method body* as the LLM's input context. In real-world projects, however, the conditions necessary to reach a target branch are often not visible within the method body itself—they are implicit in a sequence of initialization and state-setting operations that must occur *before* the method is invoked [1]. The irrelevant statements in the method body not only consume the context window but also obscure these cross-method dependencies, leaving the LLM unable to infer the required preconditions.

A Motivating Example. Consider the `addValue()` method from Apache Commons Math [15], shown in Listing 1.

Listing 1: Target branch example from Apache Commons Math

```
1 // org.apache.commons.math3.stat.descriptive.  
2 SummaryStatistics  
3 public void addValue(double value) {  
4     sumImpl.increment(value);  
5     ...  
6     if (meanImpl != mean) { // target branch  
7         meanImpl.increment(value);  
8     }  
9     ...  
10    n++;  
11 }
```

Suppose a test generation tool identifies that the branch at line 9 (`meanImpl != mean`) has not been covered and attempts to generate a test to reach it. Existing approaches typically provide the entire method body alongside CFG path information to the LLM. However, the LLM faces a critical inferential barrier: the condition `meanImpl != mean` can only be satisfied if `setMeanImpl()` was called *prior* to `addValue()`—a dependency that is entirely absent from the method body. Consequently, the LLM tends to generate a test such as:

Listing 2: Typical LLM-generated test (target branch unreachable)

```
1 // meanImpl == mean always holds; target branch is  
2 never triggered  
3 SummaryStatistics stats = new SummaryStatistics();  
4 stats.addValue(1.0);
```

This test compiles and executes without error, yet the target branch is never triggered. The correct test requires establishing the necessary precondition *before* invoking `addValue()`:

Listing 3: Correct test (target branch reachable)

```
1 // Establish precondition via setMeanImpl() before  
2 calling addValue()  
3 SummaryStatistics stats = new SummaryStatistics();  
4 stats.setMeanImpl(new CustomMeanImpl()); // makes  
5 meanImpl != mean
```

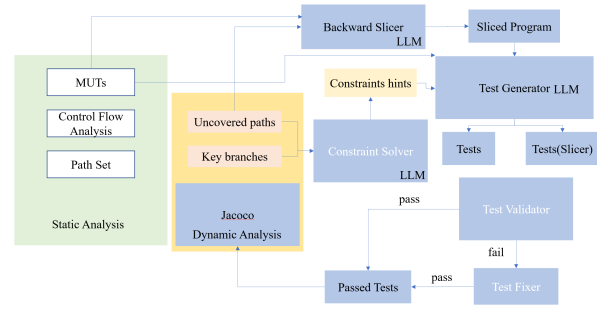


Figure 1: Overview of the coverage-guided test generation framework.

```
4 stats.addValue(1.0); // target  
   branch now reachable
```

This example illustrates two core challenges that existing approaches fail to address: (1) the method body contains numerous statements irrelevant to the target branch, which dilute the LLM's attention and impede precise reasoning; and (2) the true precondition for reaching the target branch is a cross-method dependency invisible within the method body itself.

Our Approach. To address these challenges, we propose two complementary mechanisms that reduce LLM reasoning complexity from the perspectives of *information reduction* and *condition explicitness*:

- **Backward Slicing.** Starting from the target branch condition `meanImpl != mean`, we perform backward slicing to trace all statements with data or control dependencies on `meanImpl` and `mean`, including field declarations, default assignments in the constructor, and the definition of `setMeanImpl()`. This eliminates six irrelevant `increment` calls and other unrelated branches from the context, allowing the LLM to focus on statements that govern branch reachability.
- **Constraint Hints.** Building on the slice, we explicitly provide the LLM with the path condition and how it can be established, e.g., “To trigger the target branch, `meanImpl != mean` must hold; this can be achieved by calling `setMeanImpl(customImpl)` before `addValue()`.” Rather than requiring the LLM to infer implicit cross-method dependencies from source code, this mechanism directly guides the LLM toward constructing the correct initialization sequence.

Together, these two mechanisms systematically reduce the difficulty of reasoning about complex branch conditions, enabling LLMs to generate tests that reach previously uncovered paths.

3 Methodology

3.1 Overview

COGPATH is a coverage-guided iterative framework for automated unit test generation. The overall workflow is illustrated in Figure 1. Starting from the method under test (MUT) and an initial test suite

(possibly empty), the framework performs static analysis to construct the control-flow graph and enumerate feasible execution paths. Dynamic execution feedback is then collected to identify uncovered lines and branches.

To efficiently exercise uncovered paths, COGPATH integrates two complementary mechanisms. First, a *Constraints-Hints* module derives explicit branch conditions along a selected path and exposes them as structured hints in the test generation prompt, making hidden input requirements visible to the LLM. Second, a *Backward Slicing* module is triggered when coverage improvement stagnates, extracting the minimal set of statements that influence a stubborn uncovered path and supplying them as targeted context.

Each iteration proceeds as follows. The framework selects uncovered paths, optionally derives constraint hints, and invokes the test generation LLM to produce candidate tests. These candidates are compiled and executed to validate their correctness. Failing tests are optionally forwarded to a repair phase. Successfully validated tests are incorporated into the test suite and coverage is re-measured, closing the feedback loop. The process terminates when the target coverage is reached, the iteration budget is exhausted, or coverage ceases to improve for a configurable number of iterations.

3.2 Constraints-Hints for Path Solving

For methods with complex branch logic, directly instructing an LLM to cover a specific execution path often fails because branch predicates are buried within nested conditionals, method calls, or arithmetic expressions. Consequently, the model cannot reliably infer the precise input conditions required to reach the desired path. To address this limitation, we introduce *Constraints-Hints*, a mechanism that translates implicit branch conditions into explicit and structured input requirements.

Path formulation. Let $P = \langle b_1, b_2, \dots, b_k \rangle$ denote the sequence of branch decisions along a target execution path π , where each predicate $b_i(\mathbf{x}) \in \{\text{true}, \text{false}\}$ is evaluated over the program input vector $\mathbf{x}$. Covering the path requires an input assignment $\mathbf{x}^*$ that satisfies all branch predicates along the path:

$$\mathbf{x}^* \in \mathcal{F}(\pi) = \{\mathbf{x} \mid b_1(\mathbf{x}) \wedge b_2(\mathbf{x}) \wedge \dots \wedge b_k(\mathbf{x})\}. \quad (1)$$

Rather than solving this constraint system symbolically—which can be computationally expensive and brittle under complex object states or indirect method calls—COGPATH employs an LLM-based constraint solver to approximate the feasible input space as a light-weight set of human-interpretable conditions:

$$\mathcal{C}(\pi) = \{c_j \mid c_j \text{ is a necessary condition on } \mathbf{x}\}. \quad (2)$$

The first uncovered branch along the path, denoted as b^* , is identified from the coverage report and provided to the solver as additional context, guiding the model toward the most immediate obstacle preventing the path from being executed.

Integration. The constraint-solving prompt is generated from a configurable template that incorporates the source code, the branch sequence of π , and the uncovered branch b^* . The solver produces $\mathcal{C}(\pi)$ as a list of conditions such as `paramA > 0` or `obj.status`

`== "ACTIVE"`. These conditions are appended to the test generation prompt, transforming an opaque path coverage objective into a concrete specification of required inputs.

Example. Consider the `addValue` method, which contains three conditional branches dispatching to overridable statistical implementations. An existing test suite covers only the false branches of all three predicates, meaning `meanImpl`, `varianceImpl`, and `geoMeanImpl` all refer to their default implementations. The first uncovered branch is `(meanImpl != mean)`. The constraint solver identifies that satisfying this predicate requires replacing the default implementation before invoking `addValue`. This object-state precondition is expressed as a constraint in $\mathcal{C}(\pi)$:

$$c : \text{meanImpl} \neq \text{mean}$$

which implies that a call such as `setMeanImpl(...)` must occur prior to invoking `addValue`. By making this requirement explicit, the test generation LLM can construct the correct setup sequence and cover the previously unreachable branch.

3.3 Backward Slicing for Coverage Plateaus

Despite coverage-guided iterative generation, certain branches may remain persistently uncovered, particularly in methods with high cyclomatic complexity or deeply nested control flow. In such cases, generative exploration often fails to infer the precise conditions required to execute the target path. To address this challenge, COGPATH incorporates a coverage-driven backward slicing mechanism that is activated only when coverage improvement stalls.

Relationship to prior work. Program slicing has long been used to identify the subset of program statements relevant to a particular computation [19]. Recent work such as NS-Slicer [14] demonstrates that pre-trained language models can approximate static slices by learning variable–statement dependencies. In the context of LLM-based test generation, HITS [18] applies slicing to guide test synthesis; however, its decomposition is driven by structural heuristics rather than coverage feedback, meaning the resulting segments are not targeted toward any specific uncovered branch. Furthermore, for methods with high cyclomatic complexity, the number of generated segments can grow substantially, incurring significant token overhead regardless of whether the corresponding code has already been covered. In contrast, COGPATH performs *path-aware* backward slicing triggered only when coverage improvement stalls, ensuring that each slice is both necessary and focused on the precise dependencies of the uncovered branch.

Slice extraction. Given an uncovered path π , the framework extracts the subset of statements that may influence the execution of that path:

$$\text{Slice}(P, \pi) = \{s_i \in P \mid s_i \rightsquigarrow \pi\}. \quad (3)$$

A dedicated LLM is then invoked with the uncovered code segment, the focal method source code, and dependency context from related classes. The model returns a structured response containing the relevant slice statements, required input values, necessary object states, and potential setup operations.

Test generation from slices. The extracted slice information is supplied as additional context to the test generation prompt. The LLM then generates targeted unit tests that incorporate the identified prerequisites directly. These tests are compiled, executed, and measured using the same validation pipeline as regular test candidates, allowing the framework to recover from coverage plateaus while maintaining the overall iterative workflow.

3.4 LLM-based Test Generation and Repair

The complete workflow of COGPATH is formalized in Algorithm 1. The framework accepts the source file and an initial test suite as input, and iteratively generates, validates, and repairs tests until a coverage target is met or a termination condition is triggered.

Initialization. The framework extracts test-scope dependencies, constructs the CFG and method dictionary from the source file, and measures baseline coverage on the initial suite (lines 1–4).

Path selection with Constraints-Hints. At each non-initial iteration, SELECTUNCOVEREDPATHS identifies candidate paths from the CFG that intersect with missed lines or branches, weighted by both coverage impact and visit history *pathHistory* (line 8). For each selected path π , the constraint solver derives $\mathcal{C}(\pi)$ by analyzing the branch sequence up to the first uncovered predicate b^* , and these conditions are embedded into the generation prompt (lines 9–13).

Backward slicing. When $\text{noI}ncr \geq \beta \cdot \Lambda$, indicating a coverage plateau, the backward slicer is invoked on the remaining missed code segments. It returns structured prerequisite information—required inputs, object states, and method mocks—which is used to construct an additional batch of targeted tests $\mathcal{T}_s$ (lines 16–19).

Validation and repair. Each generated test is individually inserted into the suite and executed. Failing tests are collected with their error messages and forwarded to the repair phase, which re-invokes the LLM with failure feedback to produce corrected versions. Successfully validated repaired tests are also incorporated into the suite (lines 21–32).

Termination. The loop exits when the coverage target τ is reached, the maximum iteration count *maxIter* is exhausted, or coverage fails to improve for Λ consecutive iterations.

4 EXPERIMENTS

4.1 Experimental Setup

In this section, we first introduce the construction and statistics of the evaluation dataset

Dataset. To evaluate the effectiveness of COGPATH in generating unit tests for complex methods, we utilize real-world subjects from the Defects4J (v2.0.1) benchmark [8]. We selected the latest fixed version of each subject program to ensure compatibility with modern build environments. Following prior work [7], we excluded projects that do not use Maven or rely on deprecated dependencies (i.e., Chart, Mockito, and Closure). The remaining 14 subjects are compatible with Java 8+, Maven (v3.6.3), and JUnit (v4.13.2). To specifically assess performance on logic-heavy code, we focus on complex classes rather than entire projects. We identified non-abstract, public classes containing at least one method

Algorithm 1: Iterative Test Generation with Constraints-Hints and Backward Slicing

Input: *src*: source file, *suite*: initial test suite (possibly empty)
Output: Augmented test suite with improved coverage
 $\text{deps} \leftarrow \text{EXTRACTDEPS}();$
 $\text{cfg}, \text{methodDict} \leftarrow \text{BUILDCFG}(\text{src});$
 $\text{noI}ncr \leftarrow 0, \text{iter} \leftarrow 0;$
 $\text{cov}, \text{missed} \leftarrow \text{RUNCOVERAGE}(\text{suite});$
while $\text{iter} < \text{maxIter}$ **and** $\text{noI}ncr < \Lambda$ **and** $\text{cov} < \tau$ **do**
 // Path selection and prompt construction
 if $\text{cov} = 0$ **or** $\text{iter} = 0$ **then**
 $\text{prompt} \leftarrow \text{BUILDBASELINEPROMPT}(\text{src}, \text{suite}, \text{deps});$
 ; // baseline prompt
 else
 $\Pi \leftarrow \text{SELETPATHS}(\text{cfg}, \text{methodDict}, \text{missed});$
 $\mathcal{C} \leftarrow \emptyset;$
 foreach *path* $\pi \in \Pi$ **do**
 $b^* \leftarrow \text{FIRSTUNCOVEREDBRANCH}(\pi, \text{missed});$
 $\mathcal{C}(\pi) \leftarrow \text{CONSTRAINTSOLVER}(\text{src}, \pi, b^*);$
 ; // Constraints-Hints
 end
 $\text{prompt} \leftarrow \text{BUILDPROMPT}(\text{src}, \text{suite}, \text{deps}, \Pi, \mathcal{C});$
 end
 // Test generation
 $\mathcal{T} \leftarrow \text{GENERATETESTS}(\text{prompt});$
 // Backward slicing for stubborn uncovered paths
 if $\text{noI}ncr \geq \beta \cdot \Lambda$ **then**
 $\text{slices} \leftarrow \text{BACKWARDSLICER}(\text{src}, \text{missed});$
 $\mathcal{T} \leftarrow \mathcal{T} \cup \text{GENERATEFROMSLICES}(\text{slices}, \text{suite}, \text{deps});$
 end
 // Validation and repair
 $\text{failed} \leftarrow \{\};$
 foreach *t* $\in \mathcal{T}$ **do**
 if $\text{VALIDATETEST}(t, \text{suite})$ **then** $\text{suite} \leftarrow \text{suite} \cup \{t\};$
 else $\text{failed} \leftarrow \text{failed} \cup \{t\};$
 end
 if $\text{failed} \neq \emptyset$ **then**
 $\text{suite} \leftarrow \text{suite} \cup \text{REPAIRANDVALIDATE}(\text{failed}, \text{src}, \text{suite}, \text{deps});$
 end
 // Coverage update and termination check
 $(\text{cov}', \text{missed}') \leftarrow \text{RUNCOVERAGE}(\text{suite});$
 if $\text{cov}' > \text{cov}$ **then** $\text{noI}ncr \leftarrow 0;$
 else $\text{noI}ncr \leftarrow \text{noI}ncr + 1;$
 $\text{cov} \leftarrow \text{cov}', \text{missed} \leftarrow \text{missed}', \text{iter} \leftarrow \text{iter} + 1;$
end
return *suite*;

with Cyclomatic Complexity (CC) > 10 [10]. Furthermore, we manually excluded outliers such as classes with extreme complexity (CC > 40) caused by massive switch-statement patterns and those with minimal explicit control flow due to deep inheritance. The statistics of our evaluation dataset are summarized in Table 1;

Evaluation Metrics. To evaluate the effectiveness of the generated unit tests, we primarily focus on code coverage, which is the

Table 1: Dataset Metrics with #MUTs

Identifier	Project	Version	#C	#MUTs	CC_{max}	CC_{min}	CC_{med}
Cli	Cli-40f	1.5-SNAP	2	31	11	11	11
Codec	Codec-18f	1.11-SNAP	7	78	31	11	13
Collections	Collections-28f	4.2-SNAP	5	218	32	11	16
Compress	Compress-47f	1.16-SNAP	9	65	22	11	13
Csv	Csv-16f	1.6-SNAP	3	72	25	11	14
Gson	Gson-16f	2.8.2-SNAP	4	75	37	11	22
JCore	JacksonCore-26f	2.10.0-SNAP	9	161	30	11	14
JDatabind	JacksonDatabind-112f	2.9.9	9	371	22	11	15
JXml	JacksonXml-5f	2.9.6-SNAP	4	118	29	12	22
Jsoup	Jsoup-93f	1.12.2-SNAP	8	171	22	11	12
JXPath	JXPath-22f	1.4-SNAP	12	115	32	11	13
Lang	Lang-4f	3.2-SNAP	17	586	30	11	16
Math	Math-2f	3.3-SNAP	30	452	31	11	14
Time	Time-13f	2.1	11	458	30	11	17
Total/Avg			130	2971	27.4	11.1	15.1

standard metric in automated test generation research [5, 12]. We report two specific metrics:

- **Line Coverage (LC):** The ratio of executable lines in the target class that are exercised by the generated test suite. It provides a baseline measure of how much of the implementation has been reached.
- **Branch Coverage (BC):** The ratio of executed decision outcomes (e.g., the true and false paths of an *if* statement) to the total number of possible branches. Given our focus on methods with high Cyclomatic Complexity ($CC > 10$), branch coverage is our primary metric, as it more accurately reflects the tool’s ability to navigate complex logical structures and boundary conditions.

All coverage data in our experiments are collected using the JaCoCo library, ensuring consistent and reproducible measurements across all subjects and baselines.

Research Questions. To evaluate the effectiveness and design of COGPATH, we address the following three research questions:

- **RQ1: Effectiveness Comparison.** How does COGPATH perform compared to state-of-the-art LLM-based test generation approaches in terms of line and branch coverage on complex, logic-heavy methods?
- **RQ2: Contribution of Components.** What are the individual contributions of the Backward Slicing (BS) and Constraint Solver (CS) components to the overall performance of COGPATH?
- **RQ3: Model Generalizability.** Is COGPATH’s effectiveness robust across different underlying Large Language Models, and how do their performances vary?

Baselines. To evaluate the performance of COGPATH, we compare it against three state-of-the-art (SOTA) LLM-based unit test generation frameworks. These baselines represent diverse paradigms in the field:

- **SymPrompt [14]:** A prominent prompt-engineering approach that integrates symbolic execution concepts with

LLMs. It utilizes path-oriented prompts to guide models in generating test inputs. We include SymPrompt to assess the effectiveness of COGPATH against methods relying solely on structured prompting without external constraint solving.

- **HITS [18]:** A high-performance framework emphasizing self-healing and iterative refinement. As a strong baseline for generating compilable and high-coverage tests, HITS allows us to evaluate COGPATH’s capability in navigating complex logical branches where iterative feedback is critical.
- **Panta [7]:** A SOTA tool that leverages advanced path analysis and context-aware prompting. Panta excels at capturing data dependencies within the Class Under Test (CUT). Comparing against Panta demonstrates the advantages of COGPATH’s synergy between branch selection and logical constraint satisfaction.

Due to the absence of several critical components in the official HITS repository, we faithfully re-implemented its workflow by strictly following the prompt templates defined in the original paper and adhering to the procedural logic of the available open-source fragments.

For a fair comparison, all baselines were configured using the same backbone model as COGPATH in each experiment. Specifically, RQ1 adopts **Qwen3-Coder-30B** (hereafter **QC3-30B**) as the backbone, while RQ2 adopts **Qwen3-Coder-30B** (hereafter **QC3-30B**). The use of different models across RQs reflects the distinct focuses of each evaluation: RQ1 prioritizes generation quality with a larger model, whereas RQ2 isolates the contribution of individual components under a more controlled, cost-efficient setting. Furthermore, all methods were provided with identical CUT information and dependency contexts to ensure that performance differences are attributable to the frameworks themselves rather than external information.

Contextual Information. To evaluate the inherent logic reasoning and code understanding capabilities of the models, we adopt

a restricted context strategy for all test generation tasks. Specifically, for each method under test (MUT), the prompt only includes the source code of its declaring class and the names of the associated import packages. No additional project-level information, such as dependency class implementations or global configuration files, is provided. This setup ensures that the performance of CogPATH and the baselines is assessed under a consistent and challenging information-constrained environment, reflecting a realistic scenario where cross-file dependencies may be complex or partially inaccessible.

Configurations. Our experiments employ two backbone LLMs depending on the research question. For RQ1 (effectiveness) and RQ3 (generalizability), we use **Qwen3.5-397B-A17B (QW3.5-397B)**, accessed via its official API. For RQ2 (ablation study), we adopt **Qwen3-Coder-30B (QC3-30B)**, deployed locally on a server equipped with an NVIDIA H800 GPU (80 GB); this lighter model is chosen for the ablation setting to enable a controlled, cost-efficient comparison across component variants. To assess model generalizability in RQ3, we additionally evaluate OpenAI's **GPT-5.4-Mini (GPT-5.1-Mini)** and DeepSeek's **DeepSeek-V3.1 (DS-V3.1)** via their respective official APIs.

To ensure reproducibility and minimize stochastic variance, all models are configured with a temperature of 0.2 and top- p of 1.0.

4.2 RQ1: Effectiveness of Our Approach

To answer RQ1, we compare CogPATH against three state-of-the-art LLM-based test generation frameworks—SymPrompt, HITS, and Panta—on 14 real-world Java projects from Defects4J (v2.0.1). The evaluation focuses on both **line coverage (LC)** and **branch coverage (BC)** of complex methods, with Cyclomatic Complexity $CC > 10$. Results are summarized in Table 2.

4.2.1 Overall Performance. As shown in Table 2, CogPATH consistently achieves the **highest average LC and BC** across all projects. Specifically, it attains an average BC of **44.41%**, outperforming SymPrompt (21.14%), HITS (34.24%), and Panta (32.32%). This demonstrates that the combination of **backward slicing** and **constraint hints** allows CogPATH to more effectively navigate complex control flows and satisfy cross-method preconditions, compared with baselines that rely solely on prompt engineering or CFG-based analysis.

The improvement over SymPrompt is particularly substantial (**+89.3%**), reflecting the limitation of approaches that lack explicit program analysis. Gains over HITS and Panta are smaller yet consistent (**+16.9%** and **+23.8%**, respectively), confirming that CogPATH's enhancements provide **complementary benefits** beyond existing path enumeration or context-aware prompting techniques.

4.2.2 Performance on High-Complexity Projects. The advantages of CogPATH are most pronounced on projects with high cyclomatic complexity and deeper inter-method dependencies. For example:

- **Gson** ($CC_{avg} = 23.25$, highest in the benchmark): BC of 41.24%, while SymPrompt reaches only 3.59%, highlighting the difficulty of context-unaware methods in handling complex branching logic.

- **JacksonCore** ($CC_{avg} = 16.33$): BC improves from 18.51% (HITS) to 35.54% (CogPATH), a gain of **+17 pp**, demonstrating that backward slicing effectively isolates branch-relevant statements.
- **Jsoup** ($CC_{avg} = 14.13$): BC rises from 17.36% (HITS) to 52.55% (CogPATH), a **+36.4 pp improvement**, underscoring the impact of constraint hints in constructing necessary object states to reach target branches.

These results indicate that CogPATH excels particularly in logic-heavy methods where complex object interactions and conditional dependencies are prevalent.

4.2.3 Cases of Limited Improvement. While CogPATH achieves the best average performance, there are a few projects where it does not outperform all baselines. For instance, **Collections** shows HITS achieving higher BC (70.00%) compared to CogPATH (34.63%), and **Compress** has similar trends. Manual inspection suggests that these projects contain a high proportion of branches governed by **simple value-range conditions** rather than complex cross-method states. In such cases, backward slicing occasionally removes auxiliary context that would otherwise help the LLM, slightly reducing coverage. This points to a potential area for refinement in **slice granularity selection**.

4.2.4 Summary. In summary, CogPATH demonstrates **robust effectiveness** across 14 real-world projects, achieving significant gains over state-of-the-art baselines, particularly in **high-complexity and logic-intensive methods**. The combined use of backward slicing and constraint hints enables the LLM to reach branches that are otherwise inaccessible to prompt-only or context-limited approaches.

Answer to RQ1

CogPATH outperforms all three baselines on average, achieving a BC of **44.41%**, representing improvements of **16.9%–89.3%** over SymPrompt, HITS, and Panta. Its advantage is most pronounced in projects with high cyclomatic complexity and cross-method dependencies (e.g., +36.4 pp on Jsoup, +17 pp on JacksonCore). In a minority of projects where branches depend primarily on simple value conditions, existing tools remain competitive, highlighting opportunities for further refinement in slice granularity.

4.3 RQ2: Ablation Study

To evaluate the individual contributions of Backward Slicing (BS) and Constraint Hints (CS), we conducted an ablation study with three degraded variants: **w/o BS** (removing backward slicing), **w/o CS** (removing constraint hints), and **w/o CS+BS** (removing both). All variants share the same iterative generation loop and are evaluated under an identical backbone (QC3-30B). Results are summarized in Table 3.

4.3.1 Overall Ablation Results. **Both components contribute independently.** Removing BS alone causes an average BC drop of **5.94 percentage points** (44.41% $\rightarrow$ 38.47%), while removing CS

Table 2: Coverage Across Approaches (Updated CogPATH)

Project	#C	CC_{avg}	SymPrompt		HITS		Panta		CogPATH	
			LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	2	11.00	17.93	9.02	73.50	59.58	63.11	37.98	84.32	70.73
Codec	7	16.14	29.68	24.96	65.73	60.25	60.53	54.82	76.97	69.60
Collections	5	18.60	32.63	32.42	68.23	70.00	33.44	32.40	40.46	34.63
Compress	9	15.33	11.84	11.34	31.64	25.24	34.36	28.45	36.09	29.05
Csv	3	16.67	37.69	23.79	28.95	14.73	50.98	37.99	62.27	52.29
Gson	4	23.25	13.16	3.59	31.54	21.25	21.81	15.20	63.82	41.24
JCore	9	16.33	23.62	21.44	19.85	18.51	26.24	21.97	41.72	35.54
JDatabind	9	15.33	9.99	6.06	18.33	14.44	18.17	14.34	23.35	17.85
JXml	4	21.50	17.45	18.56	18.59	18.84	31.89	23.55	20.61	17.08
Jsoup	8	14.13	24.72	19.13	27.82	17.36	49.80	39.25	66.43	52.55
JXPath	12	15.50	12.01	9.48	23.11	22.76	34.10	29.35	42.28	35.84
Lang	17	16.88	43.98	40.42	62.82	59.77	39.26	36.61	59.21	52.14
Math	30	16.47	38.55	33.70	30.87	26.38	46.40	39.16	59.60	54.02
Time	11	17.36	53.37	42.06	63.32	50.23	46.74	41.42	66.79	59.25
Average			26.19	21.14	40.31	34.24	39.77	32.32	53.14	44.41

Table 3: Ablation Study of CogPATH Components

Project	#C	CC_{avg}	CogPath		w/o BS		w/o CS+BS		w/o CS	
			LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	2	11.00	84.32	70.73	68.69	57.68	72.62	56.54	84.74	68.75
Codec	7	16.14	76.97	69.60	66.77	57.94	66.34	56.53	54.86	50.68
Collections	5	18.60	40.46	34.63	32.42	36.60	37.43	34.45	39.51	38.59
Compress	9	15.33	36.09	29.05	25.43	19.04	23.45	18.11	31.22	28.58
Csv	3	16.67	62.27	52.29	36.47	24.80	60.54	47.07	46.18	36.38
Gson	4	23.25	63.82	41.24	49.40	31.85	40.92	26.52	43.94	27.73
JacksonCore	9	16.33	41.72	35.54	45.39	39.20	22.97	21.07	30.58	27.31
JacksonDatabind	9	15.33	23.35	17.85	33.74	26.55	19.59	15.54	12.44	9.33
JacksonXml	4	21.50	20.61	17.08	35.30	26.84	26.79	23.08	23.80	18.61
Jsoup	8	14.13	66.43	52.55	69.11	53.76	59.00	47.11	61.48	46.64
JXPath	12	15.50	42.28	35.84	28.06	22.35	34.94	31.44	47.56	41.96
Lang	17	16.88	59.21	52.14	56.84	51.38	57.12	53.91	66.05	63.04
Math	30	16.47	59.60	54.02	38.60	32.66	43.23	39.90	63.42	56.61
Time	11	17.36	66.79	59.25	64.99	57.89	43.21	40.65	46.29	42.81
Average			53.14	44.41	46.51	38.47	43.44	36.57	46.58	39.79

alone results in a drop of **4.62 pp** (44.41% $\rightarrow$ 39.79%). Removing both components yields the largest degradation of **7.84 pp** (44.41% $\rightarrow$ 36.57%), confirming that BS and CS address complementary aspects of the problem: BS eliminates irrelevant context that misleads the LLM, while CS makes cross-method preconditions explicit so that the LLM can construct the required object state. Notably, the combined degradation (7.84 pp) is smaller than the sum of individual degradations (10.56 pp), suggesting that the two components partially compensate for each other when one is absent.

BS is particularly critical for context-heavy methods. The impact of BS is most pronounced in projects where method bodies contain many statements unrelated to the target branch. On *Csv*, removing BS causes BC to drop from 52.29% to 24.80% (−27.49 pp), the largest per-project degradation in the study. This aligns

with our motivation: when irrelevant statements dominate the focal method, providing the full unsliced context actively distracts the LLM from the branch-relevant logic.

CS is critical when preconditions span method boundaries. CS contributes most on projects where the target branch condition depends on prior method calls. On *JacksonDatabind*, removing CS reduces BC from 17.85% to 9.33% (−8.52 pp, a relative decline of 47.7%), indicating that without explicit constraint hints the LLM is unable to construct the object state necessary to reach the focal branch.

Anomalies and discussion. In two projects (*JacksonCore* and *Jsoup*), the w/o-BS variant achieves marginally higher coverage than the full CogPATH. We attribute this to cases where backward slicing inadvertently removes type information required for mock

construction, causing the LLM to fall back on simpler yet occasionally effective test patterns. This sensitivity to slice granularity is a known limitation and remains a direction for future refinement.

4.3.2 Iterative Coverage Improvement. Figure 2 traces branch coverage across iterations for all four configurations. COGPATH consistently achieves the highest coverage at every iteration and exhibits the steepest growth curve, demonstrating that both BS and CS contribute not only to final coverage but also to the efficiency of iterative improvement. In contrast, removing either component leads to slower convergence, and removing both produces the flattest growth trajectory, underscoring the necessity of each component for sustained progress.

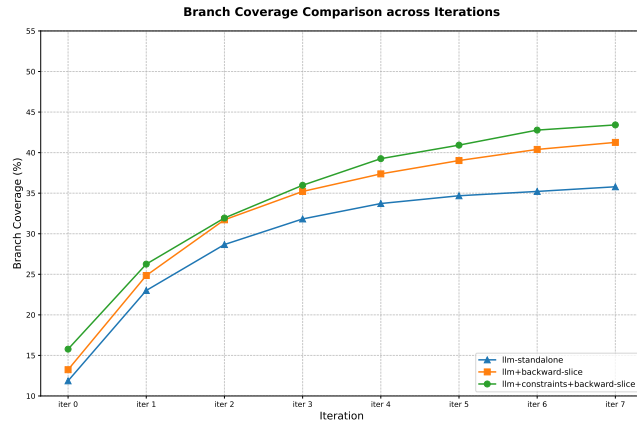


Figure 2: Branch coverage improvement over iterations for COGPATH and its three ablated variants.

Answer to RQ2

Both BS and CS make independent and complementary contributions to COGPATH. BS provides the larger share of improvement by eliminating irrelevant context (−5.94 pp when removed), while CS explicitly guides the LLM toward satisfying cross-method preconditions (−4.62 pp when removed). Removing both components degrades BC by 7.84 pp on average, with per-project drops reaching up to 27.49 pp (Csv), confirming that neither component is redundant. The iterative growth curves further show that both components contribute to the efficiency of coverage accumulation across generation rounds.

4.4 RQ3: Performance Comparison of Different LLMs

In this research question, we investigate how COGPATH generalizes across different large language models. Specifically, we evaluate the framework on four LLMs: GPT-5.1-CM, QC3-30B, QW3.5-397B, and DS-V3.1, to understand the impact of model capacity and architecture on test generation effectiveness.

Table 4 presents line coverage (LC) and branch coverage (BC) achieved by each model across 14 real-world Java projects. The

results show that COGPATH consistently achieves high coverage regardless of the underlying LLM. Notably, QW3.5-397B and DS-V3.1 attain the highest average branch coverage (83.47% and 80.12%, respectively), indicating that larger and more advanced models tend to produce more semantically accurate and coverage-effective tests. In contrast, smaller models such as QC3-30B achieve lower coverage (44.41%), highlighting the influence of model reasoning capacity on path exploration, particularly in projects with higher cyclomatic complexity.

Across all projects, COGPATH demonstrates the ability to generate tests that traverse complex execution paths and satisfy intricate object state dependencies, which often remain uncovered by smaller or less capable LLMs. The performance gap is most pronounced in projects such as Collections and JDatebind, where intricate state initialization and cross-method interactions are critical for branch reachability. This suggests that model capacity plays a significant role in LLM-based test generation, especially when reasoning about implicit preconditions and multi-step object state construction.

Answer to RQ3

COGPATH significantly outperforms state-of-the-art LLM-based test generation tools. It achieves an average branch coverage of **40.03%**, representing a **16.9% – 89.3%** improvement over existing baselines. Its advantage is particularly pronounced in projects with high cyclomatic complexity, where it successfully traverses logical paths that other tools fail to exercise.

5 DISCUSSION

COGPATH incurs additional prompting overhead due to backward slicing and constraint hint construction. On average, each focal method consumes approximately **43,000 tokens**, reflecting both the input code context and the appended constraints. Across all 3,576 requests in our evaluation, this corresponds to roughly **1.43 billion input tokens** and **12.5 million output tokens**, for a total of about **1.55 billion tokens**. The backward slicing step helps mitigate this cost by removing irrelevant statements, reducing the prompt length by roughly **25%** before constraint hints are appended. In absolute terms, a project with 130 focal methods costs about **\$100**, which is reasonable considering the resulting coverage improvements.

Token Efficiency. Despite the large aggregate token consumption, COGPATH achieves a favorable coverage-per-token ratio by focusing the LLM on relevant code slices and explicit constraints. This ensures that each token contributes meaningfully to exercising complex branches, improving the overall efficiency of test generation compared to naive full-context prompts.

Qualitative Insight. To illustrate why COGPATH outperforms baseline approaches, we examine a representative example from the *Favicon for Alibaba* project. The method generateFavicon contains a target branch condition `if (icon != null && icon.size() > 0)`, which can only be satisfied if the object is properly initialized beforehand. Baselines generate tests that invoke the focal

Table 4: Model Generalization Across Different LLMs

Project	#C	CC	GPT-5.1-CM		QC3-30B		QW3.5-397B		DS-V3.1	
			LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	2	11.00	–	–	84.32	70.73	98.82	92.63	98.82	91.93
Codec	7	16.14	–	–	76.97	69.60	98.32	95.30	93.60	88.46
Collections	5	18.60	–	–	40.46	34.63	98.16	94.43	98.02	94.20
Compress	9	15.33	–	–	36.09	29.05	86.92	80.90	81.32	76.04
Csv	3	16.67	–	–	62.27	52.29	95.71	87.42	85.70	77.53
Gson	4	23.25	–	–	63.82	41.24	92.88	87.97	92.13	82.86
JCore	9	16.33	–	–	41.72	35.54	83.25	76.77	66.62	57.76
JDataBind	9	15.33	–	–	23.35	17.85	74.93	69.81	84.06	77.44
JXml	4	21.50	–	–	20.61	17.08	62.20	63.65	66.06	65.22
Jsoup	8	14.13	–	–	66.43	52.55	95.87	87.49	87.15	79.98
JXPath	12	15.50	–	–	42.28	35.84	71.30	66.22	76.89	73.21
Lang	17	16.88	–	–	59.21	52.14	91.72	87.44	88.21	85.02
Math	30	16.47	–	–	59.60	54.02	92.96	87.47	91.69	85.41
Time	11	17.36	–	–	66.79	59.25	96.94	91.08	94.71	89.92
Average			–	–	53.14	44.41	88.57	83.47	85.89	80.12

method directly, failing to establish the required state and leaving the branch uncovered. COGPATH, by applying backward slicing, identifies the relevant state-setting call `setIconState` and provides a constraint hint specifying the precondition. The resulting test correctly initializes the object state and triggers the target branch. This example demonstrates that COGPATH’s benefits extend beyond quantitative gains: it enhances the LLM’s capacity to reason about cross-method dependencies in realistic Java programs.

Limitations. Despite its effectiveness, COGPATH has notable constraints. *Slice granularity.* For projects such as *Collections* and *Compress*, where branches depend on simple value-range conditions rather than cross-method state, the backward slice may remove auxiliary type or range information, reducing test quality. Conversely, overly coarse slices may retain irrelevant statements, introducing noise. Developing adaptive slicing strategies that adjust granularity based on branch type remains future work. *Scalability.* Backward slicing and constraint extraction introduce preprocessing overhead proportional to the inter-procedural dependency graph. In our evaluation, average preprocessing time per focal method is **1.2 s**, acceptable for medium-scale projects, though on-demand or incremental slicing may be required for industrial-scale codebases.

6 RELATED WORK

6.1 Unit Test Generation

Automated unit test generation has long been studied to reduce developer effort and improve software reliability. Existing techniques can be broadly categorized into search-based approaches and recent large language model (LLM)-based methods.

Search-based software testing (SBST) techniques construct test inputs by optimizing coverage objectives. A representative system is EvoSuite [5], which employs evolutionary algorithms to generate JUnit test suites that maximize structural coverage metrics such as branch coverage. EvoSuite adopts whole-test-suite optimization and regression assertion generation to capture program behavior

automatically. Although SBST techniques are effective at exploring input spaces, they often struggle with complex semantic dependencies and may produce tests that are difficult for developers to interpret [11].

The emergence of large language models has enabled a new class of approaches that synthesize tests directly from source code. ChatUniTest [3] leverages LLMs to generate unit tests using a focal-context mechanism that selectively exposes relevant program information to the model, combined with a generate–validate–repair workflow to improve compilation and execution success rates. SymPrompt [14] explores prompt engineering strategies that decompose test generation into structured reasoning steps aligned with program execution paths, enabling the model to better utilize type and dependency information.

Several recent studies further investigate how structured prompting can guide LLMs toward generating more effective tests. HITS [18] introduces a hierarchical prompting strategy that organizes test generation into multiple reasoning stages, allowing the model to progressively reason about program structure and test construction. This design improves the model’s ability to generate syntactically valid and semantically meaningful tests.

Other approaches integrate execution feedback to improve test reliability. CoverUp [13] and Panta [7] adopt workflows that combine LLM-based generation with compilation and execution signals to refine generated tests and improve coverage. These studies show that incorporating program feedback can significantly enhance the executability and effectiveness of generated test suites.

Another line of work focuses on practical tooling for developers. JUnitGenie [9] leverages LLMs to automatically synthesize JUnit tests from source code and project context, providing an assistant that integrates test generation into development workflows.

Despite these advances, existing approaches still exhibit important limitations. Search-based methods primarily focus on input exploration and often fail to capture semantic dependencies across methods [21], while LLM-based approaches typically rely on static prompting strategies that lack explicit reasoning about program

constraints. These limitations motivate approaches that more tightly integrate program analysis with LLM reasoning to guide test generation.

6.2 Program Slicing

Program slicing is a classical program analysis technique used to extract the subset of program statements that are relevant to a particular computation or variable [19][16]. Traditional slicing techniques rely on static or dynamic dependency analysis to identify statements that influence the value of a program element at a specific program point [16]. JavaSlicer [6], for example, performs dynamic slicing for Java programs by tracking runtime data and control dependencies, enabling developers to isolate statements that contribute to specific execution behaviors. Such slicing techniques have been widely used in debugging, program comprehension, and software testing.

Recent work has explored learning-based approaches to overcome the limitations of traditional slicing. NSSlicer [20] introduces a neural slicing framework designed for incomplete or partial programs. By leveraging neural models to predict relevant dependencies, NSSlicer can generate meaningful slices even when static analysis is insufficient or when the code lacks full contextual information. This demonstrates the potential of integrating learning-based techniques with traditional program analysis.

In the context of LLM-based unit test generation, some studies adopt simplified forms of program decomposition. For instance, HITS [18] divides complex programs into smaller code segments to reduce the cognitive load for the LLM during test generation. However, this process does not follow the traditional notion of program slicing based on data and control dependencies; instead, it primarily serves as a heuristic strategy to simplify the input context provided to the model.

In contrast, CogPATH leverages classical program analysis concepts more directly. Specifically, we perform backward slicing starting from uncovered branch conditions to identify the statements that influence the required program state. The resulting slice captures the dependencies necessary to satisfy the target branch. By providing this focused slice to the LLM, CogPATH enables the model to reason about the required preconditions and generate tests that establish the necessary object state, thereby improving branch coverage.

7 CONCLUSION

In this work, we presented CogPATH, a novel framework for automated unit test generation that leverages Backward Slicing (BS) and Constraint Hints (CS) to address the challenges of context-heavy methods and cross-method preconditions. Through an extensive empirical evaluation on a diverse set of real-world Java projects, we demonstrated that both components contribute independently and complementarily to branch coverage improvement.

Our ablation study confirmed that BS is particularly effective in eliminating irrelevant context, which can mislead LLM-based test generation in complex methods, while CS explicitly guides the

LLM to satisfy necessary preconditions that span method boundaries. The combination of BS and CS consistently achieves the highest coverage growth, outperforming variants with one or both components removed, and illustrating that neither component is redundant.

Beyond the immediate improvements in branch coverage, our analysis highlights practical insights for future research. In particular, slice granularity and context selection require careful tuning, and there are opportunities to further enhance constraint-guided LLM reasoning for complex initialization sequences.

Overall, CogPATH provides a promising direction for bridging symbolic analysis techniques with LLM-driven test generation, improving both the effectiveness and reliability of automated testing in real-world software projects. Future work includes extending the approach to handle even more complex object-oriented patterns and integrating additional dynamic program analysis to further refine constraint guidance.

References

- [1] Roberto Baldoni, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. arXiv:1610.00502 [cs.SE] <https://arxiv.org/abs/1610.00502>
- [2] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. 2008. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs.. In *OSDI*, Vol. 8. 209–224.
- [3] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. *ChatUniTest: A Framework for LLM-Based Test Generation*. arXiv:2305.04764 <http://arxiv.org/abs/2305.04764>
- [4] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Boda-pati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A Huerta, and Hao Peng. 2025. Context Length Alone Hurts LLM Performance Despite Perfect Retrieval. arXiv:2510.05381 [cs.CL] <https://arxiv.org/abs/2510.05381>
- [5] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (Szeged, Hungary) (ESEC/FSE '11)*. Association for Computing Machinery, New York, NY, USA, 416–419. doi:10.1145/2025113.2025179
- [6] Carlos Galindo, Sergio Perez, and Josep Silva. 2022. A Program Slicer for Java (Tool Paper). In *Software Engineering and Formal Methods*, Bernd-Holger Schlingloff and Ming Chai (Eds.). Springer International Publishing, Cham, 146–151.
- [7] Sijia Gu, Noor Nashid, and Ali Mesbah. 2025. LLM Test Generation via Iterative Hybrid Program Analysis. doi:10.48550/arXiv.2503.13580 arXiv:2503.13580
- [8] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (San Jose, CA, USA) (ISSTA 2014)*. Association for Computing Machinery, New York, NY, USA, 437–440. doi:10.1145/2610384.2628055
- [9] Dianshu Liao, Xin Yin, Shidong Pan, Chao Ni, Zhenchang Xing, and Xiaoyu Sun. 2025. *Navigating the Labyrinth: Path-Sensitive Unit Test Generation with Large Language Models*. doi:10.48550/arXiv.2509.23812 arXiv:2509.23812
- [10] T.J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. doi:10.1109/TSE.1976.233837
- [11] Phil McMinn. 2011. Search-Based Software Testing: Past, Present and Future. In *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. 153–163. doi:10.1109/ICSTW.2011.100
- [12] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion (Montreal, Quebec, Canada) (OOPSLA '07)*. Association for Computing Machinery, New York, NY, USA, 815–816. doi:10.1145/1297846.1297902
- [13] Juan Altmayer Pizzorno and Emery D. Berger. 2024. *CoverUp: Coverage-Guided LLM-Based Test Generation*. arXiv:2403.16218 <http://arxiv.org/abs/2403.16218>
- [14] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting Using LLM. *Proceedings of the ACM on Software Engineering* 1, FSE (July 2024), 951–971. doi:10.1145/3643769
- [15] The Apache Software Foundation. 2016. Apache Commons Math 3.6.1. <https://commons.apache.org/proper/commons-math/>. Version 3.6.1.
- [16] Frank Tip. 1995. A survey of program slicing techniques. *J. Program. Lang.* 3, 3 (1995). <http://compsinet.dcs.kcl.ac.uk/JPLP/jp030301.abs.html>

[17] Weiwei Wang, Jiyong Min, and Weijie Zou. 2026. Intelligence Degradation in Long-Context LLMs: Critical Threshold Determination via Natural Length Distribution Analysis. arXiv:2601.15300 [cs.CL] <https://arxiv.org/abs/2601.15300>

[18] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento CA USA, 2024-10-27). ACM, 1258–1268. doi:10.1145/3691620.3695501

[19] Mark Weiser. 1984. Program Slicing. *IEEE Trans. Softw. Eng.* 10, 4 (July 1984), 352–357. doi:10.1109/TSE.1984.5010248

[20] Aashish Yadavally, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2024. A Learning-Based Approach to Static Program Slicing. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (April 2024), 83–109. doi:10.1145/3649814

[21] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis. *arXiv preprint arXiv:2404.04966* (2024).

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009